

- Jeffrey -

The Sudoku Puzzle Solving Robot

<Kind of>

Wendy Stengel

Charleston Southern University Computer Science Senior Project

Advisor: Professor Underwood

20 April 2011

1 Introduction

1.1 Project Summary

This project has two main components: a C++ program that efficiently solves all 9 x 9 Sudoku puzzles and a robot capable of writing the digits 1 – 9, as well as a few letters. The robot was built using Lego Mindstorms NXT 2.0, and the code for its motion was written in NXC (Not Exactly C).

1.2 Project Purpose and Objectives

The purpose of this project was to build a robot capable of completing the following objectives:

- Use a webcam to view a Sudoku puzzle and convert the puzzle values from the image into a string
- Solve the puzzle
- Fill in the puzzle using a pen

In order to solve the puzzle (see Figure 1) correctly, the algorithm had to follow the rules of Sudoku: Using the numbers provided, fill the grid with the integers 1 to 9 such that each row, column, and box (3x3 regions bordered by thick lines) contains a single instance of all the integers from 1 to 9.

	9				6		4	
		5	3					8
				7		2		
		1		5				3
	6				9		7	
2				8	4	1		
		3		1				
8					2	5		
	5		4				8	

Figure 1 Example of a Sudoku Puzzle

2 Development

2.1 Proposed Schedule

Aug 2010	Complete Robot Prototype
Dec 2010	Complete and Test C++ SudokuSolver Code
Jan 2011	Complete and Test Code for Robot Motion
Feb 2011	Convert C++ Code to NXC
Mar 2011	Integrate SudokuSolver Code with Robot
Apr 2011	Present Robot and Graduate

2.2 Development Software and Languages

2.2.1 Microsoft Visual Studio (C++)

Microsoft Visual Studio was used when creating and testing the initial SudokuSolver code. Initially written in C++, the program was more easily tested in Visual Studio without the interference of issues arising from using a robot.

2.2.2 Briccx Command Center (NXC)

Briccx Command Center was used to convert and compile the SudokuSolver code to NXC (Not Exactly C), which was created for the Lego Mindstorms bricks. Briccx was only able to identify compile time errors, not run any type of simulation. This became an issue when trying to integrate the software with the robot.

3 Design

3.1 Software

The algorithm to solve a Sudoku puzzle was created to function the way most people do when presented with a puzzle: it evaluates which values are possible for each cell and reduces those values as other cells are solved. The main data structure for the program is a two-dimensional array of Sudoku structs, which contain the possible values, actual value, and block location for each cell. Figure 2 shows the array locations for each of the cells in a 9 x 9 puzzle.

(0,0)	(0,1)	(0,2)	(0,3)	(0,4)	(0,5)	(0,6)	(0,7)	(0,8)
(1,0)	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(1,7)	(1,8)
(2,0)	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)	(2,7)	(2,8)
(3,0)	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)	(3,7)	(3,8)
(4,0)	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)	(4,7)	(4,8)
(5,0)	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)	(5,7)	(5,8)
(6,0)	(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)	(6,7)	(6,8)
(7,0)	(7,1)	(7,2)	(7,3)	(7,4)	(7,5)	(7,6)	(7,7)	(7,8)
(8,0)	(8,1)	(8,2)	(8,3)	(8,4)	(8,5)	(8,6)	(8,7)	(8,8)

Figure 2 Sudoku Array Locations

When determining which value was appropriate for any given cell, the following decision tables and trees were used:

Number Exists in Row	Y	Y	Y	Y	N	N	N	N
Number Exists in Column	Y	Y	N	N	Y	Y	N	N
Number Exists in Square	Y	N	Y	N	Y	N	Y	N
Number is a Possible Value								X
Number is not a Possible Value	X	X	X	X	X	X	X	

Figure 3 Possible Value Decision Table

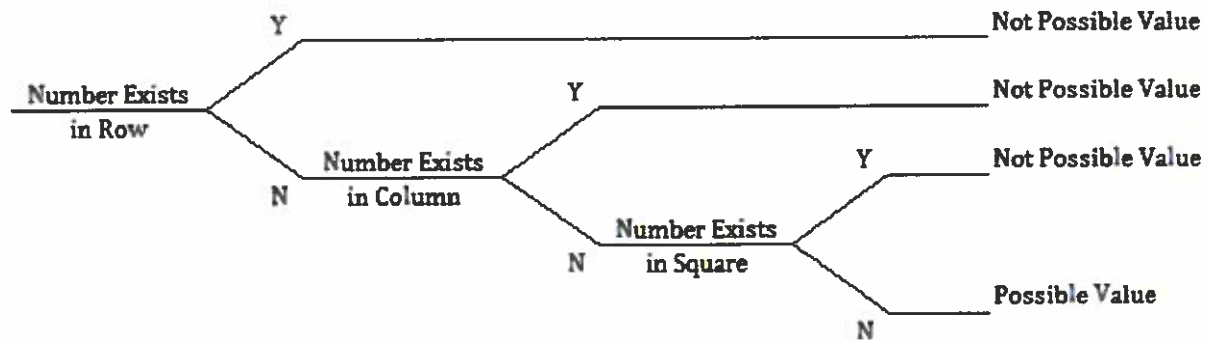


Figure 4 Possible Value Decision Tree

Only Possible Value	Y	Y	Y	Y	Y	Y	Y	Y	N	N	N	N	N	N	N	N	N
Only Occurrence in Row	Y	Y	Y	Y	N	N	N	N	Y	Y	Y	Y	N	N	N	N	N
Only Occurrence in Column	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	Y	Y	N	N	N
Only Occurrence in Square	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	Y	N	N
Actual Value	X								X								
Not Actual Value										X	X	X	X	X	X	X	X
Error		X	X	X	X	X	X	X									

Figure 5 Actual Value Decision Table

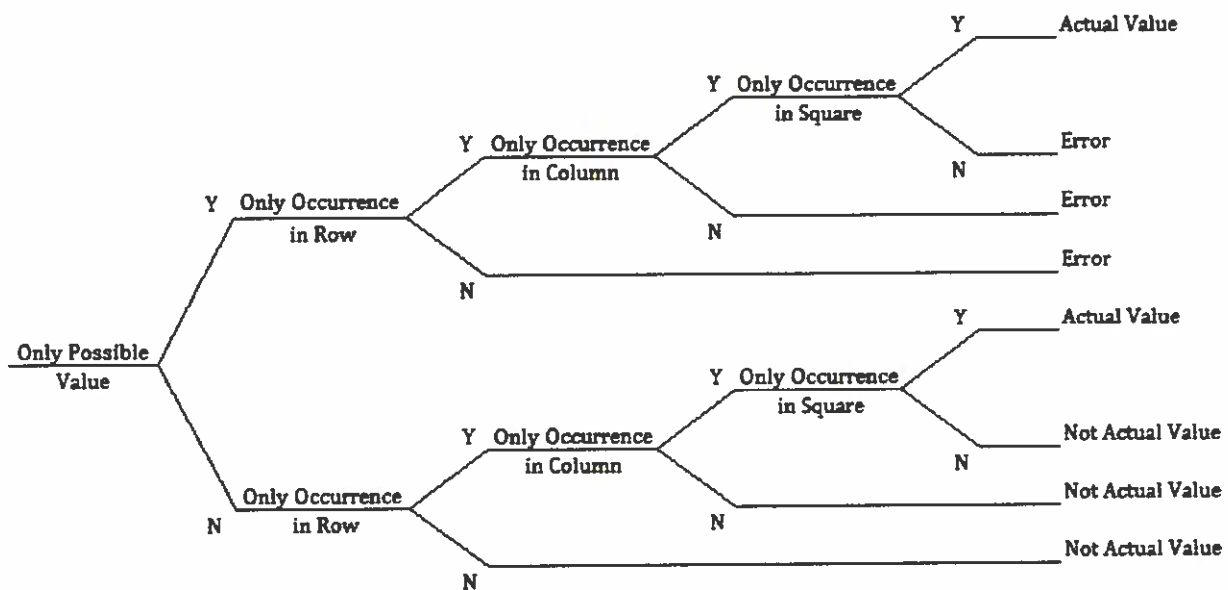


Figure 6 Actual Value Decision Tree

An additional feature of the program is the ability to guess when there are no logical steps left and the puzzle is incomplete. As guessing has no place in a logic puzzle as it is illogical, the algorithm had to be modified to include a data capture and restore ability should a guess be incorrect. The guesser() function is called if no cells have been solved within three passes of the code. The puzzle array is then copied into a three-dimensional Sudoku array for restoration, if needed. The first cell that contains two possible values is then assigned the first of those values and the program again attempts to solve the puzzle. This is done as many times as necessary for the puzzle to be solved. Should the puzzle be unable

to solve and there are cells with no possible values and no actual value, then the previous guess is retracted and the second possible value from the previously used cell is assigned.

Appendix B includes the full code for the SudokuSolver Program.

3.2 Hardware

The robot was designed to be able to move forward and backward, with a jointed arm that moves left and right, and up and down. There were some significant issues with the weight of the arm causing too much drag on the pen, so the robot (formerly known as Frank) was redesigned and rebuilt several times. Appendix A contains images of the current physical configuration of Jeffrey.

4 Problems and Solutions

4.1 Motor Precision

A major issue preventing the robot from writing legibly is a lack of motor precision. The call to move a motor includes a movement speed and the movement angle. Unfortunately, those movements are extremely inconsistent, even within the same run.

To resolve this issue, the arm was rebuilt to be lighter and use a second, smaller gear to require a greater angle of rotation to turn a smaller distance. This was able to reduce the error a small amount, and was also successfully used for the forward and reverse motor motion. The code for the robot motion can be found in its entirety in Appendix C.

4.2 Hardware / Software Integration

Integrating the converted Sudokusolver code with the robot became an insurmountable issue. The Briccx compiler did not flag any errors, but the robot kept throwing a "File Error!" message. Given the fact that there is no interface for viewing the code or specific errors as the program is running on the robot, it is extremely difficult to determine where the error is located.

One error was resolved by changing the array of structs to separate arrays, but the code was still not functional. The end result is a very fast C++ SudokuSolver program and a robot that is capable of writing numbers.

Appendix A: Images of Jeffrey the Robot

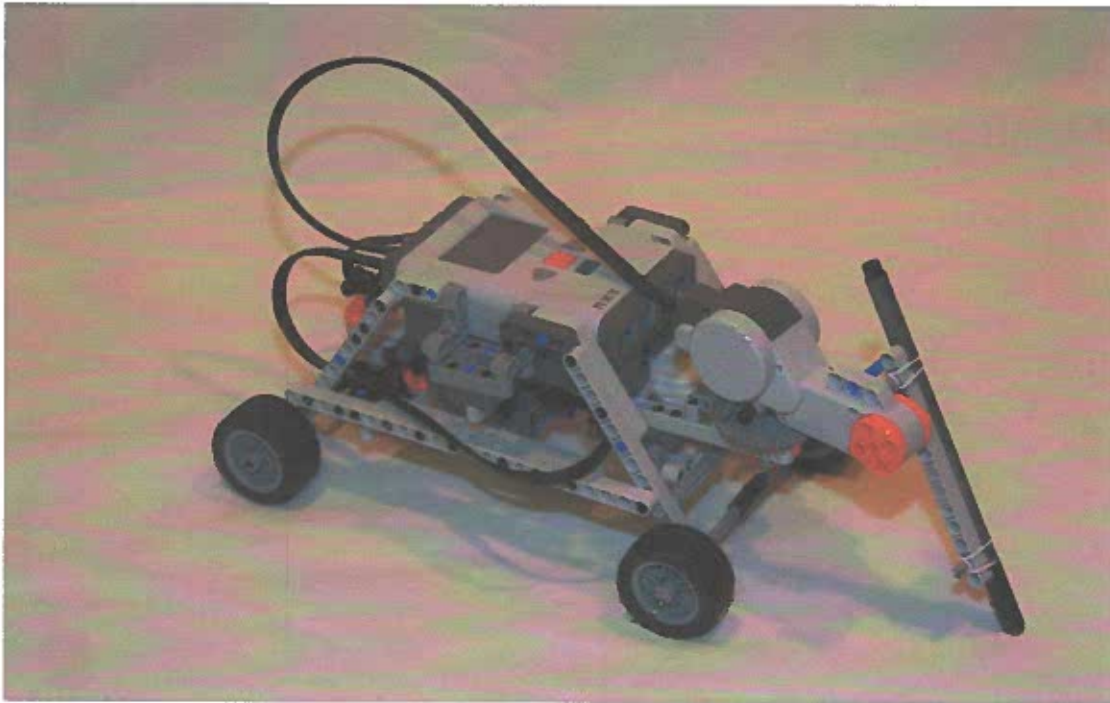


Figure 2 Jeffrey the Robot

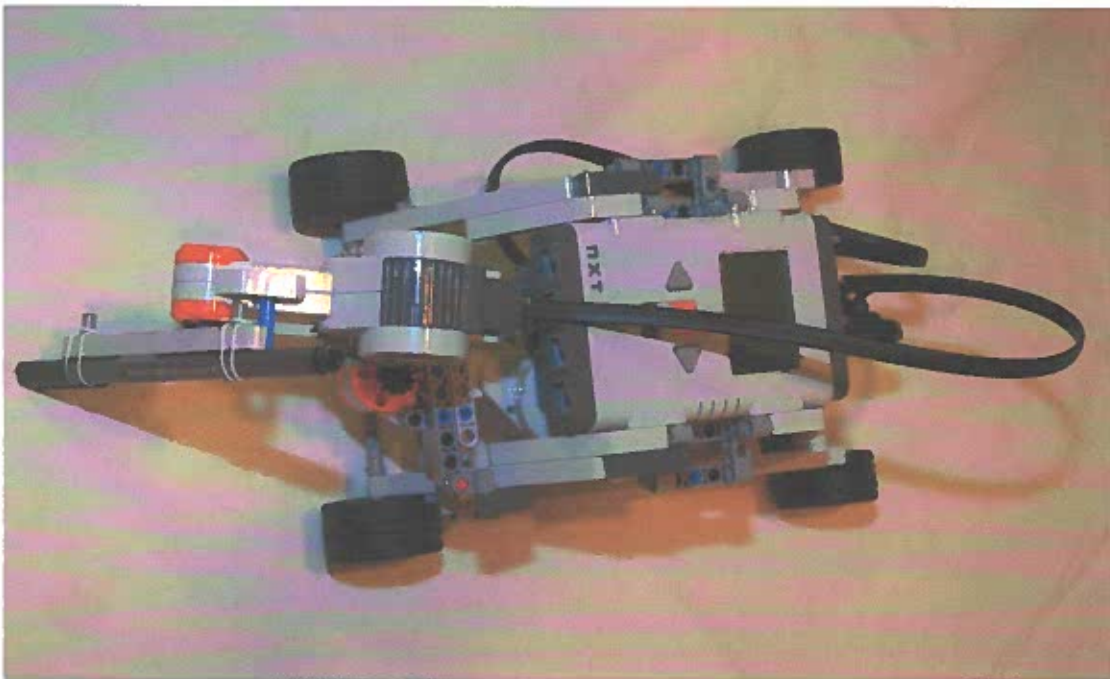


Figure 3 Top View

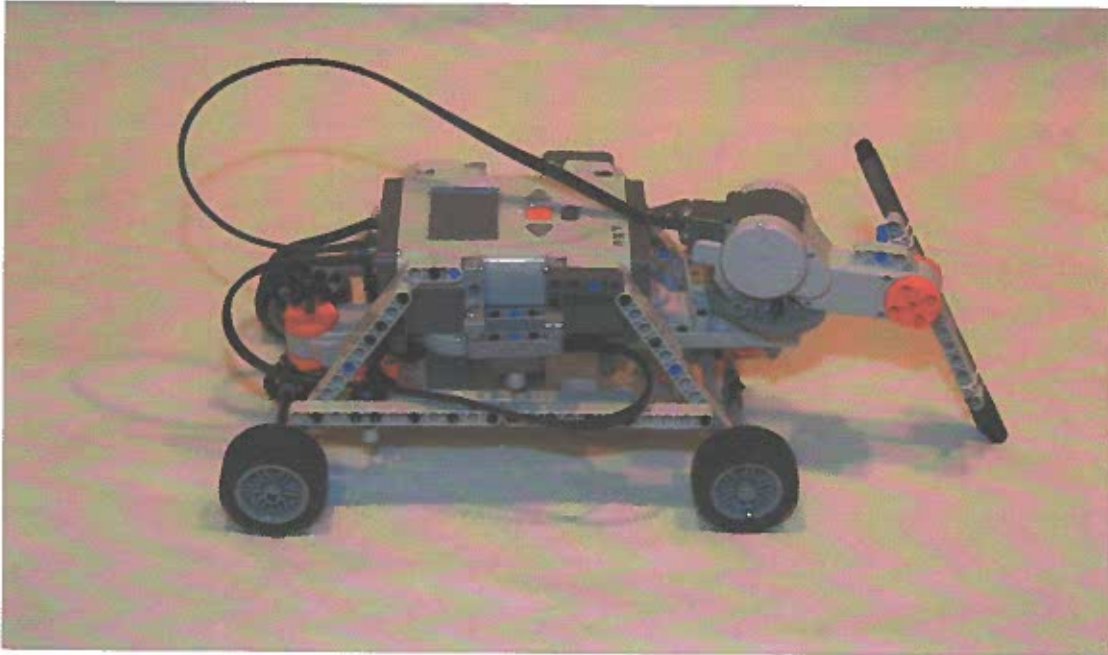


Figure 4 Right Side View



Figure 5 Rear View



Figure 6 Left Side View

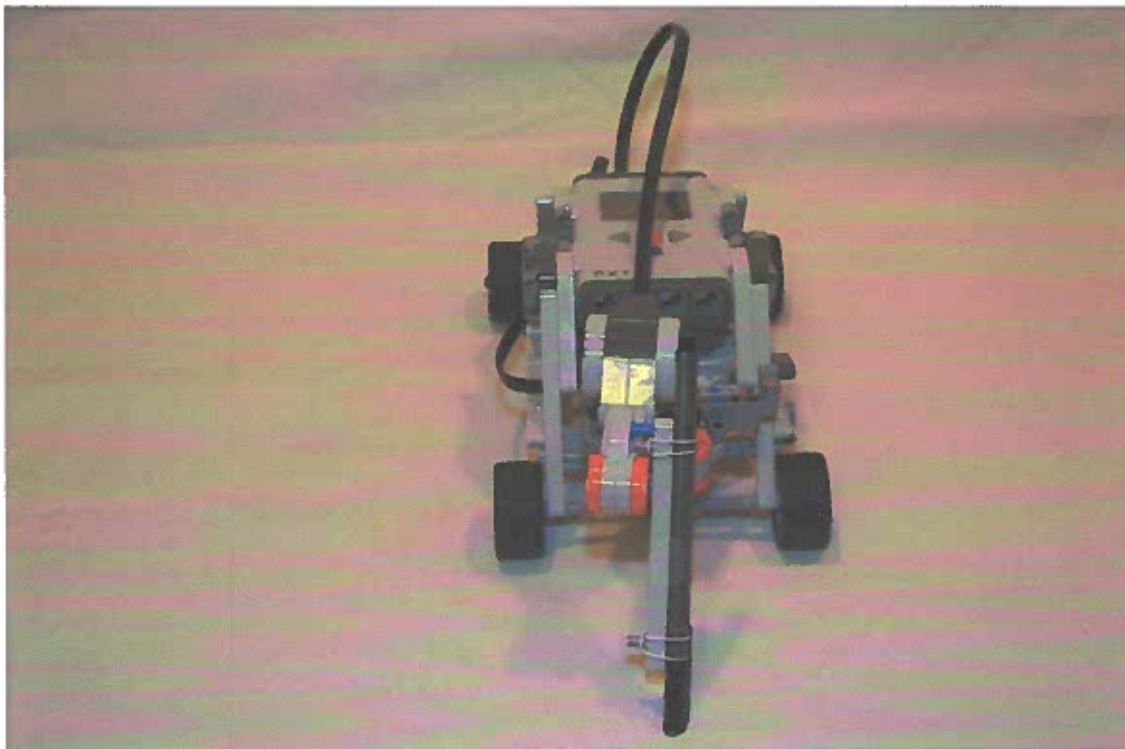


Figure 7 Front View

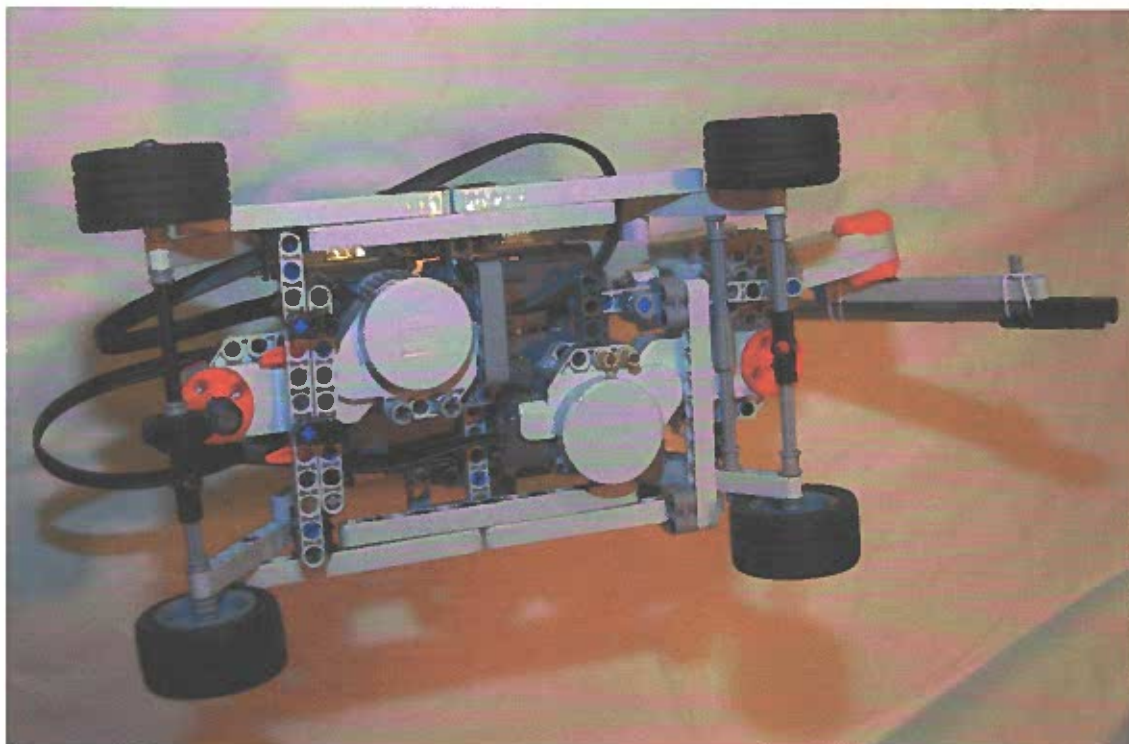


Figure 8 Bottom View

Appendix B: C++ SudokuSolver Code

B.1 main.cpp

```
#include "SudokuSolver.h"
#include <iostream>
using namespace std;

void main(){
    bool solved = false;
    SudokuSolver frank;
    frank.getVals();
    frank.findPoss();
    frank.print();
    while (!solved)
        solved = frank.findActual();
    frank.print();
}
```

5.1

B.2 SudokuSolver.h

```
#include <string>
using namespace std;

class SudokuSolver {
public:
    SudokuSolver(string input);
    SudokuSolver();
    void findPoss();
    void print();
    void getVals();
    bool findActual();

private:
    //functions
    void setBlocks(int blk, int r, int c);
    void findPoss(char num, int r, int c, int blk);
    bool checkCell(int r, int c);
    bool checkRow(int r, int c, int i);
    bool checkCol(int r, int c, int i);
    bool checkBlk(int r, int c, int i);
    void copy();
    void retract();
    void guesser();

    //variables & data structures
    struct Sudoku {
        char val;
        int block;
        string possVals;
    };
    struct BlockContents {
        int row;
```

```

        int column;
    };
    Sudoku cells[9][9];
    BlockContents blocks[9][9];
    string values;
    string possibles;
    Sudoku guess[9][9][9];
    int depth;    //location in guess array
};

```

B.3 SudokuSolver.cpp

```

#include "SudokuSolver.h"
#include <iostream>
using namespace std;

SudokuSolver::SudokuSolver(string input) {
    values = input;
    possibles = "123456789";
    depth = 0;
}

SudokuSolver::SudokuSolver() {
    //easy puzzles
    //values =
    "9820000430070009156510940005201070903100000670402060810005201361950004002600
    00759";
    //values =
    "1000405702540908003001650040006013906105090420754020004003570010010204590680
    10007";
    //hard puzzles
    values =
    "00023000000600080001000000000080060030090000000000070120001003000000070500000
    06000";
    //values =
    "0000000237008000001000009000500320000000007000000600000007018000300000500009
    00000";
    //very hard puzzles (require guessing)
    //values =
    "000700200000000050701000000000002301070800000005000000000008190000000006300002
    00000";
    //values =
    "70009000000000000030000000502000009100005030000008000000001072000080607000500
    00000";
    //values =
    "0000069002000000703000000000100007500003080000000200000002500030090006000007
    00000";
    possibles = "123456789";
    depth = 0;
}

void SudokuSolver::setBlocks(int blk, int r, int c) {    //loads blocks
    array with corresponding cell locations
    //current locations in each block

```

```

        static int zero = 0, one = 0, two = 0, three = 0, four = 0, five = 0,
six = 0, seven = 0, eight = 0;
        int locale = 0;    //if can't use pointer, increment in switch statement
after assignment
        switch (blk) {
case 0:
        locale = zero;
        zero++;
        break;
case 1:
        locale = one;
        one++;
        break;
case 2:
        locale = two;
        two++;
        break;
case 3:
        locale = three;
        three++;
        break;
case 4:
        locale = four;
        four++;
        break;
case 5:
        locale = five;
        five++;
        break;
case 6:
        locale = six;
        six++;
        break;
case 7:
        locale = seven;
        seven++;
        break;
case 8:
        locale = eight;
        eight++;
        break;
        }
        blocks[blk][locale].row = r;
        blocks[blk][locale].column = c;
    }

void SudokuSolver::print() {
    for (int r = 0; r < 9; r++) { //r is row
        for (int c = 0; c < 9; c++) { //c is column
            if (cells[r][c].val == '0') cout << " ";
            else cout << cells[r][c].val;
            if (c == 2 || c == 5) cout << "|";
            if (c == 8) cout << "\n";
            if ((r == 2 && c == 8) || (r == 5 && c == 8)) cout << "----
+---+---\n";
        }
    }
}

```

```

        else if (j == 8) { //number is not in other cells - assign
            cells[r][c].val = cells[r][c].possVals[i];
            cells[r][c].possVals = "";
            findPoss(cells[r][c].val, r, c, cells[r][c].block);
            return true;
        }
    }
    return false;
}

bool SudokuSolver::checkBlk(int r, int c, int i) {
    for (int j = 0; j < 9; j++) { //check against block's cells
        int location1 = r * 10 + c;
        int blockRow = blocks[cells[r][c].block][j].row;
        int blockCol = blocks[cells[r][c].block][j].column;
        int location2 = blockRow * 10 + blockCol;
        if (location1 != location2 &&
            cells[blockRow][blockCol].possVals.find(cells[r][c].possVals[i]) !=
            string::npos) break;
        else if (j == 8) { //number is not in other cells - assign
            cells[r][c].val = cells[r][c].possVals[i];
            cells[r][c].possVals = "";
            findPoss(cells[r][c].val, r, c, cells[r][c].block);
            return true;
        }
    }
    return false;
}

bool SudokuSolver::findActual() {
    int completed = 0; //counts number of solved cells
    static int count = 0;
    for (int r = 0; r < 9; r++) { //r is row
        for (int c = 0; c < 9; c++) { //c is column
            if (cells[r][c].val == '0') {
                for (int i = 0; i < cells[r][c].possVals.length();
i++) {
                    if (checkCell(r, c)) break; //check if only
possible value in cell
                    if (checkRow(r, c, i)) break; //check if only
possible value in row
                    if (checkCol(r, c, i)) break; //check if only
possible value in column
                    if (checkBlk(r, c, i)) break; //check if only
possible value in block
                }
                count ++;
            }
            else {
                completed++;
                count = 0;
            }
        }
    }
    if (completed == 81) return true;
    return false;
}

```

```

void SudokuSolver::copy() { //save current puzzle configuration
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            guess[depth][r][c].val = cells[r][c].val;
            guess[depth][r][c].possVals = cells[r][c].possVals;
        }
    }
    depth++;
}

void SudokuSolver::retract() {
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            cells[r][c].val = guess[depth][r][c].val;
            cells[r][c].possVals = guess[depth][r][c].possVals;
        }
    }
    depth--;
}

void SudokuSolver::guesser() { //if cannot solve puzzle with actual
logic
    static Sudoku guessee; //structure to hold data from guess
    static int row, column;
    bool flag = false;
    //determine if a new guess is needed or last guess should be retracted
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (cells[r][c].possVals == "" && cells[r][c].val == '0') {
                flag = true;
                break;
            }
        }
        if (flag) break;
    }

    //code for retraction
    if (flag) {
        retract();
        guessee.val = '0';
        guessee.possVals = cells[row][column].possVals;
        cells[row][column].val = guessee.possVals[1];
        findPoss(cells[row][column].val, row, column,
cells[row][column].block);
        return;
    }

    //code for guess
    copy();
    for (int r = 0; r < 9; r++) {
        for (int c = 0; c < 9; c++) {
            if (cells[r][c].possVals.length() > 2) continue;
            guessee.val = '0';
            guessee.possVals = cells[r][c].possVals;
            row = r;
            column = c;

```

```
cells[r][c].val = cells[r][c].possVals[0];  
findPoss(cells[r][c].val, r, c, cells[r][c].block);  
return;
```

```
}
```

```
}
```

```
}
```

Appendix C: Robot Movement Code (NXC)

```
// Motor A
void moveUp() { RotateMotor(OUT_A, 100, -300); }
void moveDown() { RotateMotor(OUT_A, 100, 300); }

// Motor B
void lower() { RotateMotor(OUT_B, 5, 29); }
void raise() { RotateMotor(OUT_B, 5, -30); }

// Motor C
void moveLeft() { RotateMotor(OUT_C, 10, 27); }
void moveRight() { RotateMotor(OUT_C, 10, -27); }

void printOne() {
    moveRight(); lower(); moveDown(); moveDown(); raise(); moveUp(); moveUp();
}

void printTwo() {
    lower(); moveRight(); moveDown(); moveLeft(); moveDown(); moveRight(); raise(); moveUp();
    moveUp();
}

void printThree() {
    lower(); moveRight(); moveDown(); moveDown(); moveLeft(); raise(); moveUp(); lower(); moveRight();
    raise(); moveUp();
}

void printFour() {
    lower(); moveDown(); moveRight(); raise(); moveUp(); lower(); moveDown(); moveDown(); raise();
    moveUp(); moveUp();
}

void printFive() {
    moveRight(); lower(); moveLeft(); moveDown(); moveRight(); moveDown(); moveLeft(); raise();
    moveUp(); moveUp(); moveRight();
}

void printSix() {
    moveRight(); lower(); moveLeft(); moveDown(); moveDown(); moveRight(); moveUp(); moveLeft();
    raise(); moveUp(); moveRight();
}

void printSeven() {
    lower(); moveRight(); moveDown(); moveDown(); raise(); moveUp(); moveUp();
}
```

```

void printEight() {
    lower(); moveDown(); moveDown(); moveRight(); raise(); moveUp(); moveUp(); moveLeft(); lower();
    moveRight(); moveDown(); moveDown(); raise(); moveUp(); lower(); moveLeft(); raise(); moveUp();
    moveRight();
}

void printNine() {
    moveRight(); lower(); moveLeft(); moveDown(); moveRight(); raise(); moveUp(); lower(); moveDown();
    moveDown(); raise(); moveUp(); moveUp();
}

void printP() {
    moveDown(); moveDown(); lower(); moveUp(); moveUp(); moveRight(); moveDown(); moveLeft();
    raise(); moveUp(); moveRight();
}

void printA() {
    lower(); moveDown(); moveDown(); raise(); moveUp(); moveUp(); lower(); moveRight();
    moveDown(); moveDown(); raise(); moveUp(); lower(); moveLeft(); raise(); moveUp(); moveRight();
}

void printM() {
    lower(); moveDown(); moveDown(); raise(); moveUp(); moveUp(); lower(); moveRight();
    moveDown(); moveDown(); raise(); RotateMotor(OUT_C, 7, 13); moveUp(); moveUp(); lower();
    moveDown(); moveDown(); raise(); RotateMotor(OUT_C, 7, -13); moveUp(); moveUp();
}

void printE() {
    moveRight(); lower(); moveLeft(); moveDown(); moveDown(); moveRight(); raise(); moveUp();
    lower(); moveLeft(); raise(); moveUp(); moveRight();
}

void nextRow() {
    RotateMotor(OUT_C, 20, 100); RotateMotor(OUT_A, 100, 800);
}

void printNumbers() {
    printOne(); RotateMotor(OUT_C, 10, -10);
    printTwo(); RotateMotor(OUT_C, 10, -10);
    printThree(); nextRow();
    printFour(); RotateMotor(OUT_C, 10, -10);
    printFive(); RotateMotor(OUT_C, 10, -10);
    printSix(); nextRow();
    printSeven(); RotateMotor(OUT_C, 10, -10);
    printEight(); RotateMotor(OUT_C, 10, -10);
    printNine(); RotateMotor(OUT_C, 20, 115); RotateMotor(OUT_A, 100, 800);
    //print "PASS ME"
}

```

```

printP(); RotateMotor(OUT_C, 10, -10);
printA(); RotateMotor(OUT_C, 10, -10);
printFive(); RotateMotor(OUT_C, 10, -10);
printFive(); nextRow();
printM(); RotateMotor(OUT_C, 10, -10);
printE(); nextRow(); moveDown(); moveDown();
}

```

```

task main() {
    //PlayFileEx(filename, volume, 0); //sound 0 - 4
    //OnFwd(OUT_A, 75);
    // OnFwd(OUT_C, 75);
    // Wait(Random(600));
    // OnRev(OUT_AC, 75);
    // Wait(4000);

    printNumbers();
    Off(OUT_ABC);
}

```

Appendix D: Failed Robot Code (NXC)

```
//data structures and variables
//*****
//sudoku cells
    char cellVal[9][9];
    char cellBlock[9][9];
    string cellPossVals[9][9];
//block contents
    char blockRow[9][9];
    char blockColumn[9][9];

    string values;
    string possibles;
    char zero = 0, one = 0, two = 0, three = 0, four = 0, five = 0, six = 0, seven = 0, eight = 0;
//*****

// Motor A
void moveUp() { RotateMotor(OUT_A, 100, -300); }
void moveDown() { RotateMotor(OUT_A, 100, 300); }

// Motor B
void lower() { RotateMotor(OUT_B, 5, 28); }
void raise() { RotateMotor(OUT_B, 5, -28); }

// Motor C
void moveLeft() { RotateMotor(OUT_C, 10, 20); }
void moveRight() { RotateMotor(OUT_C, 10, -20); }

int length(string temp) {      //returns length of string
    int i = 0;
    while (temp[i] != 0) i++;
    return i;
}

int find(string temp, char num) { //return index of location in string
    int size = length(temp);
    for (int i = 0; i < size; i++)
        if (temp[i] == num) return i;
    return -1;
}

void erase(string temp, int index) {
    int size = length(temp);
    for (int i = index; i <= size; i++) temp[i] = temp[i+1];
}
```

```

void SudokuSolver() {
    //easy puzzles
    //values =
    "982000043007000915651094000520107090310000067040206081000520136195000400260000759";
    //values =
    "100040570254090800300165004000601390610509042075402000400357001001020459068010007";
    //hard puzzles
    values =
    "00023000000600080001000000000080060030090000000000701200010030000007050000006000";
    //values =
    "000000023700800000100000900050032000000000700000060000000701800030000050000900000";
    //very hard puzzles (require guessing)
    //values =
    "0007002000000005070100000000000230107080000000500000000000819000000000630000200000";
    //values =
    "70009000000000000003000000050200000910000503000000800000000107200008060700050000000";
    //values =
    "000006900200000070300000000010000750000308000000020000000250003009000600000700000";
    possibles = "123456789";
}

```

```

void setBlocks(int blk, int r, int c) { //loads blocks array with corresponding cell locations
    //current locations in each block
    int locale = 0;
    switch (blk) {
case 0:
        locale = zero;
        zero++;
        break;
case 1:
        locale = one;
        one++;
        break;
case 2:
        locale = two;
        two++;
        break;
case 3:
        locale = three;
        three++;
        break;
case 4:
        locale = four;
        four++;
        break;
case 5:
        locale = five;

```

```

        five++;
        break;
case 6:
    locale = six;
    six++;
    break;
case 7:
    locale = seven;
    seven++;
    break;
case 8:
    locale = eight;
    eight++;
    break;
    }
    blockRow[blk][locale] = r;
    blockColumn[blk][locale] = c;
}

void getVals() { //initialize sudoku puzzle array
    int strCount = 0;
    for (int r = 0; r < 9; r++) { //r is row
        for (int c = 0; c < 9; c++) { //c is column
            cellVal[r][c] = values[strCount];
            if (r >= 0 && r <= 2) { //blocks 0 - 2
                if (c >= 0 && c <= 2) { setBlocks(0, r, c); cellBlock[r][c] = 0; }
                else if (c >= 3 && c <= 5) { setBlocks(1, r, c); cellBlock[r][c] = 1; }
                else if (c >= 6 && c <= 8) { setBlocks(2, r, c); cellBlock[r][c] = 2; }
            }
            else if (r >= 3 && r <= 5) { //blocks 3 - 5
                if (c >= 0 && c <= 2) { setBlocks(3, r, c); cellBlock[r][c] = 3; }
                else if (c >= 3 && c <= 5) { setBlocks(4, r, c); cellBlock[r][c] = 4; }
                else if (c >= 6 && c <= 8) { setBlocks(5, r, c); cellBlock[r][c] = 5; }
            }
            else if (r >= 6 && r <= 8) { //blocks 6 - 8
                if (c >= 0 && c <= 2) { setBlocks(6, r, c); cellBlock[r][c] = 6; }
                else if (c >= 3 && c <= 5) { setBlocks(7, r, c); cellBlock[r][c] = 7; }
                else if (c >= 6 && c <= 8) { setBlocks(8, r, c); cellBlock[r][c] = 8; }
            }
            strCount++;
            if (cellVal[r][c] == '0') cellPossVals[r][c] = possibles;
            else cellPossVals[r][c] = "";
        }
    }
}

void findPoss(char num, int r, int c, int blk) { //removes found value from
row's/column's/block's cells[].possVals; called from findActual()

```

```

        for (int i = 0; i < 9; i++) {
int findex = find(cellPossVals[r][i], num);
            if (findex != NULL) erase(cellPossVals[r][i], findex);
            findex = find(cellPossVals[i][c], num);
            if (findex != NULL) erase(cellPossVals[i][c], findex);
            int bRow = blockRow[blk][i];
            int bCol = blockColumn[blk][i];
            findex = find(cellPossVals[bRow][bCol], num);
            if (findex != NULL) erase(cellPossVals[bRow][bCol], findex);
        }
    }

void findPossFirst() { //removes initial puzzle values from row's/column's/block's cells[].possVals
    for (int r = 0; r < 9; r++) { //r is row
        for (int c = 0; c < 9; c++) { //c is column
            if (cellVal[r][c] != '0') { //if cell value has been determined
                for (int i = 0; i < 9; i++) { //traverse row/column/block
int findex = find(cellPossVals[r][i], cellVal[r][c]);
                    if (findex != -1) erase(cellPossVals[r][i], findex);
                    findex = find(cellPossVals[i][c], cellVal[r][c]);
                    if (findex != -1) erase(cellPossVals[i][c], findex);
                                int bRow = blockRow[cellBlock[r][c]][i];
                                int bCol = blockColumn[cellBlock[r][c]][i];
                                findex = find(cellPossVals[bRow][bCol], cellVal[r][c]);
                    if (findex != -1) erase(cellPossVals[bRow][bCol], findex);
                }
            }
        }
    }
}

bool checkCell(int r, int c) {
    if (cellVal[r][c] == '0' && length(cellPossVals[r][c]) == 1) {
        string poss = cellPossVals[r][c];
        cellVal[r][c] = poss[0];
        cellPossVals[r][c] = "";
        findPoss(cellVal[r][c], r, c, cellBlock[r][c]);
        return true;
    }
    return false;
}

bool checkRow(int r, int c, int i) {
    for (int j = 0; j < 9; j++) { //check against row's cells
        string poss = cellPossVals[r][c];
        int findex = find(cellPossVals[j][c], poss[i]);
        if (j != r && findex != NULL) break;
        else if (j == 8) { //number is not in other cells - assign

```

```

    poss = cellPossVals[r][c];
    cellVal[r][c] = poss[i];
        cellPossVals[r][c] = "";
        findPoss(cellVal[r][c], r, c, cellBlock[r][c]);
        return true;
    }
}
return false;
}

bool checkCol(int r, int c, int i) {
    for (int j = 0; j < 9; j++) { //check against column's cells
        string poss = cellPossVals[r][c];
        int findex = find(cellPossVals[r][j], poss[i]);
        if (j != c && findex != NULL) break;
        else if (j == 8) { //number is not in other cells - assign
            poss = cellPossVals[r][c];
            cellVal[r][c] = poss[i];
            cellPossVals[r][c] = "";
            findPoss(cellVal[r][c], r, c, cellBlock[r][c]);
            return true;
        }
    }
    return false;
}

bool checkBlk(int r, int c, int i) {
    for (int j = 0; j < 9; j++) { //check against block's cells
        int location1 = r * 10 + c;
        int bRow = blockRow[cellBlock[r][c][j];
        int bCol = blockColumn[cellBlock[r][c][j];
        int location2 = bRow * 10 + bCol;
        string poss = cellPossVals[r][c];
        int findex = find(cellPossVals[bRow][bCol], poss[i]);
        if (location1 != location2 && findex != NULL) break;
        else if (j == 8) { //number is not in other cells - assign
            poss = cellPossVals[r][c];
            cellVal[r][c] = poss[i];
            cellPossVals[r][c] = "";
            findPoss(cellVal[r][c], r, c, cellBlock[r][c]);
            return true;
        }
    }
    return false;
}

bool findActual() {
    int completed = 0; //counts number of solved cells

```

```

        for (int r = 0; r < 9; r++) { //r is row
            for (int c = 0; c < 9; c++) { //c is column
                if (cellVal[r][c] == '0') {
                    int size = length(cellPossVals[r][c]);
                    for (int i = 0; i < size; i++) {
                        if (checkCell(r, c)) break; //check if only possible value in
cell
                        if (checkRow(r, c, i)) break; //check if only possible value in
row
                        if (checkCol(r, c, i)) break; //check if only possible value in
column
                        if (checkBlk(r, c, i)) break; //check if only possible value in
block
                    }
                }
                else completed++;
            }
        }
        if (completed == 81) return true;
        return false;
    }

//print number functions
//*****
void printOne() {
    moveRight(); lower(); moveDown(); moveDown(); raise(); moveUp(); moveUp();
}

void printTwo() {
    lower(); moveRight(); moveDown(); moveLeft(); moveDown(); moveRight(); raise(); moveUp();
moveUp();
}

void printThree() {
    lower(); moveRight(); moveDown(); moveDown(); moveLeft(); raise(); moveUp(); lower(); moveRight();
raise(); moveUp();
}

void printFour() {
    lower(); moveDown(); moveRight(); raise(); moveUp(); lower(); moveDown(); moveDown(); raise();
moveUp(); moveUp();
}

void printFive() {
    moveRight(); lower(); moveLeft(); moveDown(); moveRight(); moveDown(); moveLeft(); raise();
moveUp(); moveUp(); moveRight();
}

```

```

void printSix() {
    moveRight(); lower(); moveLeft(); moveDown(); moveDown(); moveRight(); moveUp(); moveLeft();
    raise(); moveUp(); moveRight();
}

void printSeven() {
    lower(); moveRight(); moveDown(); moveDown(); raise(); moveUp(); moveUp();
}

void printEight() {
    lower(); moveDown(); moveDown(); moveRight(); moveUp(); moveUp(); moveLeft(); raise();
    moveDown(); lower(); moveRight(); raise(); moveUp();
}

void printNine() {
    moveRight(); lower(); moveLeft(); moveDown(); moveRight(); raise(); moveUp(); lower(); moveDown();
    moveDown(); raise(); moveUp(); moveUp();
}

void print() {
    for (int r = 0; r < 9; r++) { //r is row
        for (int c = 0; c < 9; c++) { //c is column
            switch(cellVal[r][c]) {
                case '1':
                    printOne();
                    break;
                case '2':
                    printTwo();
                    break;
                case '3':
                    printThree();
                    break;
                case '4':
                    printFour();
                    break;
                case '5':
                    printFive();
                    break;
                case '6':
                    printSix();
                    break;
                case '7':
                    printSeven();
                    break;
                case '8':
                    printEight();
                    break;
                case '9':

```

```

        printNine();
        break;
    }
    RotateMotor(OUT_C, 10, -10);
}
RotateMotor(OUT_C, 20, 300); RotateMotor(OUT_A, 100, 900);
}
}
//*****

task main(){
    bool solved = false;
    SudokuSolver();
    getVals();
    findPossFirst();
    while (solved == false)
        solved = findActual();
    print();
}

```